

DAYJOY AI

Enterprise Platform — Setup & Production Roadmap

A 22-step mentor-style guide taking the platform from code-complete to production-deployed, with verified real integrations.

Scope • Environment → Database → Backend → RAG → AI/LLM → Vapi Voice → WhatsApp → Web Chat → Customer/Distributor/Employee Portals → Admin → n8n → Notifications → Monitoring → Security → Testing → Staging → Pilot → Production → Verification

Audience • Platform owner, DevOps engineer, integration partner

Output • Production-ready Dayjoy AI Enterprise stack on AWS ap-south-1

How to use this guide

This document is the single execution playbook for taking the Dayjoy AI Enterprise codebase from its current code-complete state to a verified production deployment. It is written as a mentor walking beside you: each step opens with the goal, lists the concrete commands and configuration changes you must run, flags the common failure modes, and closes with an explicit acceptance checklist. Do not skip ahead — every step depends on the artifacts produced by the previous one.

The platform ships as a pnpm/Next.js workspace with five frontend apps (admin-dashboard, customer-portal, distributor-portal, employee-portal, website-chat), a NestJS backend, a TypeScript RAG pipeline, a Vapi voice module, a WhatsApp integration, shared packages, database migrations, deployment manifests, monitoring, and a 189-document docs tree. The codebase is approximately 70% production-ready: the FastAPI voice assistant is the most complete module, while several NestJS services and Vapi tools still return mocks. This roadmap addresses every gap.

Iron rule: Do NOT enter Step 3 (Database) until Step 1 (Environment & Secrets) and Step 2 (Local Infrastructure) are signed off. A misconfigured secret or a missing Redis instance will silently corrupt your seed data and produce hours of debugging.

Pre-flight inventory

#	Checkpoint	Owner	Done
1	Verified dayjoy-ai-enterprise-final.zip extracted to ~/dayjoy-ai-enterprise	You	
2	Node 22+, pnpm 9+ installed (node -v, pnpm -v)	You	
3	Docker Desktop / Rancher Desktop running (docker info)	You	
4	AWS CLI v2 configured (aws sts get-caller-identity)	You	
5	Domain(s) purchased: admin.dayjoy.ai, customer.dayjoy.ai, api.dayjoy.ai	Ops	
6	OpenAI API key with \$200+ credit (gpt-4o + text-embedding-3-small)	You	
7	Vapi account + verified phone number (vapi.ai)	Voice	
8	WhatsApp Business Cloud API account (Meta Business)	WhatsApp	
9	Twilio account with India DLT-approved sender ID	Telephony	
10	GitHub repo + branch protection on main	DevOps	

STEP 01

Environment & Secrets

Establish a single source of truth for every environment variable and never commit a real secret.

Begin by copying `.env.example` to `.env` at the repository root. The example file documents every variable the backend, RAG, voice, and WhatsApp modules expect. Walk through each section and fill in real values, starting with the two secret fields that must be cryptographically strong: **JWT_SECRET** and **SESSION_SECRET**. Generate them with `openssl rand -hex 32` — never reuse a value across

environments and never shorten below 32 characters.

Once your local `.env` is populated, mirror every variable into AWS Secrets Manager under the secret ID `dayjoy/prod` (matches `AWS_SECRET_MANAGER_SECRET_ID`). The External Secrets Operator (installed in Step 17) will sync this secret into Kubernetes, so the same identifier must be used everywhere. For local development you can keep `.env` in the repo root; for staging and production, `.env` must be git-ignored and only the AWS Secrets Manager copy is used.

```
# Generate strong secrets
openssl rand -hex 32 # → JWT_SECRET
openssl rand -hex 32 # → SESSION_SECRET

# Copy template and edit
cp .env.example .env
nano .env

# Mirror to AWS Secrets Manager (run from deployment/scripts/)
aws secretsmanager create-secret --name dayjoy/prod \
  --secret-string file:///./prod-secrets.json --region ap-south-1
```

Common failure: leaving `JWT_SECRET=replace-with-min-32-char-secret-key-in-production` in `.env`. The backend boots fine but every issued token becomes trivially forgeable. Add a CI gate that fails the build if `JWT_SECRET` matches the example value.

Acceptance checkpoints

#	Checkpoint	Owner	Done
1	<code>.env</code> created with strong <code>JWT_SECRET</code> , <code>SESSION_SECRET</code> , <code>OPENAI_API_KEY</code>	DevOps	
2	AWS Secrets Manager secret <code>dayjoy/prod</code> exists and contains all required keys	DevOps	
3	GitHub branch protection rejects any commit that touches <code>.env</code>	DevOps	
4	<code>CORS_ORIGINS</code> list matches the three planned subdomains	Backend	

STEP 02

Local Infrastructure

Stand up PostgreSQL with pgvector, Redis, MinIO, and MailHog locally via Docker Compose.

The repository ships with a production-shaped `docker - compose . yml` that brings up Postgres 16 with the `pgvector` extension, Redis 7, MinIO (S3-compatible object storage for RAG document uploads), and MailHog (for capturing outbound email in development). Bring everything up before you touch the database or backend code; the backend's Prisma client will fail to connect if Postgres is not yet listening on 5432, and the JWT blocklist will throw on every request if Redis is not yet accepting connections on 6379.

```
cd dayjoy-ai-enterprise
docker compose up -d
docker compose ps # all services 'healthy'
docker compose logs -f postgres # wait for 'database system is ready to accept connections'
```

Verify the pgvector extension is enabled by connecting to the database and running `SELECT * FROM pg_extension WHERE extname = 'vector';`. If the row is missing, run `CREATE EXTENSION IF NOT EXISTS vector;` as the superuser. MinIO should expose its console on `http://localhost:9001` — log in with the credentials from `.env` and create a bucket named `dayjoy-rag` (this name is referenced by the RAG ingestion module).

Common failure: running `docker compose up` without the `-d` flag, then accidentally killing the foreground process with `Ctrl-C`, leaving half the services running. Always use `-d` and verify with `docker compose ps`.

Acceptance checkpoints

#	Checkpoint	Owner	Done
1	<code>docker compose ps</code> shows postgres, redis, minio, mailhog as healthy	DevOps	
2	<code>pg_extension</code> row for 'vector' exists in <code>dayjoy_ai</code> database	Backend	
3	MinIO bucket <code>dayjoy-rag</code> created and writable	Backend	
4	<code>redis-cli ping</code> returns PONG	Backend	

STEP 03

Database

Apply all four SQL migrations, generate the Prisma client, and seed reference data.

The database layer uses both raw SQL migrations (under `database/migrations/`) and a unified Prisma schema (`database/prisma/schema.prisma`) with 63 models and 28 enums. The migrations create the tables and indexes; the Prisma client provides the typed query layer consumed by every backend service. Run them in order: first the SQL migrations (which create enums and constraints), then Prisma generate (which reads the schema and emits the client), then the seed script (which inserts tenants, the `SUPER_ADMIN` user, RBAC roles, and the demo distributor).

```
# 1. Apply SQL migrations in order
psql -d dayjoy_ai -f database/migrations/001_initial_schema.sql
psql -d dayjoy_ai -f database/migrations/002_business_tables.sql
psql -d dayjoy_ai -f database/migrations/003_ai_tables.sql
psql -d dayjoy_ai -f database/migrations/004_rag_chunks_pgvector.sql

# 2. Generate Prisma client
pnpm db:generate

# 3. Seed reference data
pnpm db:seed

# 4. Verify
pnpm db:studio # opens Prisma Studio on http://localhost:5555
```

After seeding, open Prisma Studio and verify the **Tenant** table has one row (the default tenant), the **User** table has the `SUPER_ADMIN` user with a bcrypt hashed password, and the **Role** and **Permission** tables are populated. Copy the default tenant's UUID into `DEFAULT_TENANT_ID` in your `.env` — every multi-tenant API call requires this header.

Common failure: running `pnpm db:generate` before the SQL migrations. The Prisma schema references enums (`OrderStatus`, `ConversationStatus`, etc.) that only exist after the migrations run. Generate will succeed (Prisma only reads the schema file) but every query against those tables will fail at runtime.

Acceptance checkpoints

#	Checkpoint	Owner	Done
1	All 4 migrations applied without errors (psql \dt shows 60+ tables)	Backend	
2	Prisma client generated (node_modules/.prisma/client exists)	Backend	
3	Seed script completed; SUPER_ADMIN user can log in	Backend	
4	DEFAULT_TENANT_ID in .env matches the seeded tenant UUID	Backend	

STEP 04

Backend

Boot the NestJS backend, verify `/health` returns 200, and smoke-test the auth flow.

With the database seeded and infrastructure up, start the backend in development mode. The `pnpm --filter backend dev` command launches NestJS with hot-reload on port 3000. Watch the console for the *Nest application successfully started* message; if you see a Prisma connection error, double-check `DATABASE_URL` in `.env` and that the postgres container is healthy. Once booted, the first endpoint to verify is `GET /health` — it should return a JSON envelope with success: `true` and a status object showing database, redis, and OpenAI connectivity.

```
# Start backend (port 3000)
pnpm --filter backend dev

# In another terminal – verify health
curl http://localhost:3000/health | jq

# Smoke-test auth: login as SUPER_ADMIN
curl -X POST http://localhost:3000/api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"admin@dayjoy.ai","password":"ChangeMe!123"}' | jq
```

The login response should include `accessToken`, `refreshToken`, and a user object with the `SUPER_ADMIN` role. Store the access token in a shell variable for the next smoke tests: `export TOKEN=<paste-token-here>`. Then hit a protected endpoint such as `GET /api/users` with `-H "Authorization: Bearer $TOKEN"` and confirm you receive the paginated user list. A 403 here means the RBAC guard is not picking up the role — verify the `UserRole` join table was seeded.

Acceptance checkpoints

#	Checkpoint	Owner	Done
1	Backend boots without errors on port 3000	Backend	
2	GET /health returns success:true with all subsystems healthy	Backend	
3	POST /api/auth/login returns access + refresh token for SUPER_ADMIN	Backend	
4	GET /api/users with Bearer token returns paginated list	Backend	

STEP 05

RAG + Knowledge

Ingest source documents, generate embeddings, and verify retrieval quality.

The RAG pipeline lives under `rag/` and is responsible for chunking source documents, generating OpenAI embeddings, storing them in `pgvector`, and retrieving the most relevant chunks for any given query. The reference knowledge base ships in `packages/knowledge-base/` — six documents covering the product catalog, distributor policy, returns SOP, voice agent playbook, ingredient compliance, and regional pricing. Ingest them all in one pass using the `rag ingest` CLI.

```
# Ingest all source documents
pnpm --filter rag ingest -- --source packages/knowledge-base/

# Verify chunk count
psql -d dayjoy_ai -c "SELECT COUNT(*) FROM rag_chunks;"

# Smoke-test retrieval
curl -X POST http://localhost:3000/api/knowledge/query \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"query":"What is the return policy for opened supplements?"}' | jq
```

A successful retrieval returns the top-5 chunks with cosine similarity scores above 0.78, plus a grounded answer synthesized by the LLM. If the similarity scores are all below 0.5, you likely forgot to generate embeddings during ingest — re-run with the `--embed` flag. If the answer is hallucinated despite good retrieval, check the `RAG Integration Prompt` in the AI Management view of the admin dashboard — the citation rules must be present.

Quality gate: Run the RAG evaluation suite at `testing/rag/retrieval-accuracy.test.ts` before moving on. The minimum acceptable `precision@5` is 0.85; if your score is lower, your chunking strategy needs tuning (try 800-token chunks with 100-token overlap).

Acceptance checkpoints

#	Checkpoint	Owner	Done
1	All 6 reference documents ingested; rag_chunks table has 1500+ rows	RAG	
2	Embeddings generated (1536-dim vectors present in rag_chunks.embedding)	RAG	
3	Retrieval smoke test returns chunks with similarity > 0.78	RAG	
4	RAG evaluation suite passes with precision@5 ≥ 0.85	RAG	

STEP 06

AI / LLM

Wire the AI conversation service to OpenAI, enable function-calling, and validate tool selection.

The AI module (backend/ai/) orchestrates conversations between end-users and the LLM. In its current state, ai/conversations.service.ts persists user messages but does not actually call the LLM — that wiring is the deliverable of this step. Open backend/ai/llm.gateway.ts and verify that OPENAI_API_KEY, OPENAI_MODEL (default gpt-4o), and OPENAI_API_BASE are read from the config service. If you are using Azure OpenAI or a proxy like LiteLLM, set the base URL and API version accordingly.

Next, enable function-calling by registering the eight tools defined in backend/ai/tools.service.ts: search_knowledge, search_products, customer_lookup, distributor_lookup, create_lead, book_appointment, create_support_ticket, and human_transfer. Each tool's execute() method must call the corresponding backend service rather than returning a mock — the mock implementations are flagged in the PRODUCTION_READINESS_ROADMAP.md and must be replaced. Once wired, smoke-test a multi-turn conversation that triggers at least one tool call.

```
# Smoke-test AI conversation with tool calling
curl -X POST http://localhost:3000/api/ai/conversations \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"channel":"website"}' | jq # → returns conversationId

curl -X POST http://localhost:3000/api/ai/conversations/$CID/messages \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"role":"user","content":"Show me the price of the wellness bundle."}' | jq
```

Acceptance checkpoints

#	Checkpoint	Owner	Done
1	POST /api/ai/conversations creates a conversation row in the database	AI	
2	POST /messages triggers an OpenAI completion (visible in server logs)	AI	
3	Function-calling works: 'price of wellness bundle' triggers search_products	AI	
4	AI evaluation suite passes (response-accuracy, tool-selection, latency)	AI	

STEP 07

Vapi Voice AI

Create a Vapi assistant, register the webhook, and complete a test inbound call.

Vapi is the voice channel. Begin in the Vapi dashboard by creating a new assistant named **Sarah** and importing the prompt configuration from `vapi/assistants/sarah.config.json`. The prompt references the RAG knowledge base and the eight function-calling tools, so make sure those are wired first (Step 6). Purchase or port a phone number in Vapi (India numbers require DLT approval — budget 3-5 business days). Configure the webhook URL to point to your publicly accessible backend:
`https://api.dayjoy.ai/api/voice/webhook`.

For local development, expose the backend with ngrok: `ngrok http 3000` and use the resulting HTTPS URL as the webhook target. The webhook handler (`vapi/webhooks/handler.ts`) must verify the Vapi signature header on every request — bypass the check only in explicit dev mode, never in production. Once the webhook is registered, dial the Vapi number from your phone and verify: (a) Sarah answers within 2 seconds, (b) she can answer a knowledge-base question, (c) she can call the `search_products` tool, and (d) the call appears in the Recent Calls table of the admin dashboard's Voice AI view.

```
# Expose backend for webhook testing
ngrok http 3000 # copy the https URL

# Set webhook URL in Vapi dashboard → Assistant → Webhook
# URL: https://<ngrok-id>.ngrok.io/api/voice/webhook

# After test call – verify call log
curl http://localhost:3000/api/voice/calls?limit=5 \
  -H "Authorization: Bearer $TOKEN" | jq
```

Acceptance checkpoints

#	Checkpoint	Owner	Done
1	Vapi assistant 'Sarah' created with prompt + tool config imported	Voice	
2	Webhook URL registered; signature verification enabled (no dev bypass)	Voice	
3	Inbound test call: Sarah answers, answers KB question, calls a tool	Voice	
4	Call appears in <code>/api/voice/calls</code> with recording URL and transcript	Voice	

STEP 08

WhatsApp AI

Configure the Meta WhatsApp Business Cloud API and exchange a test message.

WhatsApp integration uses Meta's official Cloud API (no on-premise client needed). Begin in the Meta Business Manager: create a WhatsApp Business Account, register an app, and obtain the `WHATSAPP_PHONE_NUMBER_ID`, `WHATSAPP_BUSINESS_ACCOUNT_ID`, and a permanent `WHATSAPP_ACCESS_TOKEN` (system user token, not a temporary one). Add these to `.env` and to the AWS Secrets Manager secret. Configure the webhook URL to `https://api.dayjoy.ai/api/whatsapp/webhook` and subscribe to the messages field; Meta will send a verification challenge that your webhook must echo back.

Test by sending a WhatsApp message from your personal number to the business number. Your backend should receive the webhook, route the message through the AI conversation service (reusing the same conversation logic as the website chat), and respond with an AI-generated reply within 5 seconds. If the reply takes longer than 15 seconds, Meta will time out and the user will see an error — profile the latency and consider switching to a streaming model or pre-generating common replies. Verify that the conversation appears in the admin dashboard's CRM view with the source labeled *WhatsApp*.

Acceptance checkpoints

#	Checkpoint	Owner	Done
1	Meta WhatsApp Business app created; access token has permanent scope	WhatsApp	
2	Webhook subscribed and verified (Meta dashboard shows green checkmark)	WhatsApp	
3	Inbound message triggers AI reply within 5 seconds	WhatsApp	
4	Conversation visible in admin dashboard with source=WhatsApp	WhatsApp	

STEP 09

Website AI Chat

Embed the chat widget on a test page and verify end-to-end messaging.

The website-chat app (apps/website-chat/) is a standalone Next.js PWA that can be embedded as a widget on any marketing site or served as a full-page chat experience. Build it with `pnpm --filter website-chat build` and deploy the static export to your CDN. The widget communicates with the backend via the same `/api/ai/conversations` endpoints used by the other channels — no separate API surface is needed.

Embed the widget on a test HTML page using the snippet provided in `apps/website-chat/EMBED.md`. Verify: (a) the widget renders in the bottom-right corner, (b) opening it creates a new conversation, (c) typing a message and pressing Enter returns an AI reply, (d) the conversation persists across page reloads via the `conversationId` stored in `localStorage`, and (e) the conversation appears in the admin dashboard with `source=Website`.

Acceptance checkpoints

#	Checkpoint	Owner	Done
1	website-chat app builds without errors (<code>pnpm --filter website-chat build</code>)	Frontend	
2	Widget embeds on test page; renders in bottom-right corner	Frontend	
3	End-to-end message round-trip works; conversation persists across reloads	Frontend	
4	Conversations visible in admin dashboard with source=Website	Frontend	

STEP 10

Customer Portal

Adapt the admin-dashboard scaffold into a customer-facing portal with login, orders, and support.

The customer-portal app (apps/customer-portal/) currently ships as a placeholder. The fastest path to production is to clone the admin-dashboard scaffold, strip out the admin-only views (AI Management, Knowledge Base, System Config), and replace them with customer-facing views: My Orders, My Profile, Support Tickets, and a Chat with Sarah widget that reuses the website-chat component. Authentication uses the same JWT flow but scopes the user to the CUSTOMER role — the backend RBAC guard will reject any attempt to access admin endpoints.

Acceptance checkpoints

#	Checkpoint	Owner	Done
1	customer-portal app boots; login flow works for CUSTOMER role users	Frontend	
2	My Orders view shows orders placed by the logged-in customer	Frontend	
3	Support ticket creation works (POST /api/tickets)	Frontend	
4	Chat widget embedded; AI replies visible	Frontend	

STEP 11

Distributor Portal

Add distributor-specific views: team management, commission tracking, and bulk ordering.

Distributors need a richer portal than customers: they manage a downstream team of sub-distributors, track commissions, place bulk orders at distributor pricing, and view sales analytics. Clone the admin-dashboard scaffold again, scope authentication to the DISTRIBUTOR role, and add four views: Team Tree (downline distributors), Commissions (monthly statement + payout history), Bulk Order (CSV upload + cart), and Analytics (their territory's sales charts, reusing the Recharts components from the admin dashboard). The backend already exposes the necessary endpoints under /api/distributors/* — verify they are RBAC-guarded to DISTRIBUTOR.

Acceptance checkpoints

#	Checkpoint	Owner	Done
1	distributor-portal app boots; login works for DISTRIBUTOR role	Frontend	
2	Team Tree view loads downline distributors recursively	Frontend	
3	Bulk order CSV upload creates multiple OrderItem rows in one transaction	Frontend	
4	Commissions view shows correct monthly payout based on distributor tier	Frontend	

STEP 12

Employee Portal

Build an internal employee portal with lead assignment, ticket queue, and call logs.

Employees (sales reps, support agents, ops staff) need a workflow-oriented portal: their day starts with a queue of assigned leads, open tickets, and recent calls. Clone the admin-dashboard scaffold, scope to EMPLOYEE role, and add three primary views: My Leads (kanban-style board with Hot/Warm/Cold columns, drag-to-reassign), Ticket Queue (sorted by SLA breach risk), and Call Logs (filterable by direction, outcome, and duration). Reuse the DataTable, Pill, and Meter components from the kit. The backend already exposes /api/leads?assignee=me and /api/tickets?assignee=me — verify these return only the logged-in

employee's items.

Acceptance checkpoints

#	Checkpoint	Owner	Done
1	employee-portal app boots; login works for EMPLOYEE role	Frontend	
2	My Leads view shows only leads assigned to the logged-in user	Frontend	
3	Ticket queue sorts by SLA breach risk (soonest first)	Frontend	
4	Call logs view filters correctly by direction and outcome	Frontend	

STEP 13

Admin Dashboard

Verify the admin dashboard's 15 views, fix non-working buttons, and connect to live backend data.

The admin dashboard is the most mature frontend app and the one you have already previewed in this session. It ships with 15 views grouped into Overview, AI Platform, Business, and Administration. In its shipped state, several buttons were no-ops (Quick Actions, Export, Upload Document, End Call, workflow pause/resume, security toggles) and four navigation items showed a Pro Locked page. The fixes applied in this session wire every button to a real handler with toast feedback; the next step is to replace the mock data in `src/data/mock.ts` with live API calls via `src/lib/api.ts`.

For each view, replace the imported mock array with a `useQuery` hook from `@tanstack/react-query`. Example: the dashboard view's `kpis` array should be replaced with `const { data: kpis } = useQuery({ queryKey: ['dashboard-kpis'], queryFn: () => api.get('/analytics/dashboard') })`. Add loading skeletons using the existing `GlassCard` component, and error states using the `InlineAlert` component. Once all 15 views are wired to live data, the dashboard is feature-complete.

Acceptance checkpoints

#	Checkpoint	Owner	Done
1	All 15 sidebar views render without console errors	Frontend	
2	All previously no-op buttons fire toast notifications (this session's fix)	Frontend	
3	Mock data replaced with live API calls; loading + error states handled	Frontend	
4	Dark/light theme toggle persists across page reloads	Frontend	

STEP 14

n8n Automation

Deploy n8n, import the eight reference workflows, and verify trigger → action execution.

n8n is the visual workflow engine that handles event-driven automation: lead capture, welcome emails, order confirmations, shipping notifications, appointment reminders, ticket auto-close, daily memory cleanup, and conversation summarization. Deploy n8n via Docker Compose (add a service to the existing `docker-compose.yml`) or as a Helm chart in your Kubernetes cluster. Once running, import the eight workflow JSON files from `automation/workflows/` and configure the credentials for each external service (SMTP, Twilio, Meta WhatsApp, Google Calendar).

For each workflow, verify the trigger fires and the action executes end-to-end. The most important workflow to test is *Lead Capture & Assignment*: create a new lead via the API, confirm n8n receives the webhook, scores the lead, assigns it to the right employee (round-robin or territory-based), sends a welcome WhatsApp message, and creates a follow-up task. If any step fails, n8n's execution log will show exactly where — fix the credential or the node configuration and re-run.

Acceptance checkpoints

#	Checkpoint	Owner	Done
1	n8n deployed and accessible at https://n8n.dayjoy.ai	Automation	
2	All 8 workflows imported and activated	Automation	
3	Lead Capture workflow fires on POST /api/leads; assignment + welcome sent	Automation	
4	Daily Memory Cleanup cron runs at 02:00 IST without errors	Automation	

STEP 15

Notifications + Email + Calendar

Wire email (SES/SendGrid), SMS (Twilio), push (FCM), and calendar (Google) providers.

The notifications module (backend/notifications/) currently logs notifications but does not actually send them. Wire four providers: **Email** via AWS SES (preferred) or SendGrid — verify domain ownership and DKIM signing; **SMS** via Twilio with an India DLT-approved sender ID; **Push** via Firebase Cloud Messaging — register the FCM server key in .env and add a device-token table to the database; **Calendar** via Google Calendar API — create an OAuth2 service account, share the booking calendar with it, and store the credentials JSON in AWS Secrets Manager.

Add a notification_preferences table to the database so users can opt out of specific channels (e.g., disable SMS but keep email). The notifications service should respect these preferences before sending. Smoke-test each channel: trigger a welcome email on user creation, an SMS on order shipment, a push on ticket update, and a calendar event on appointment booking.

Acceptance checkpoints

#	Checkpoint	Owner	Done
1	SES domain verified; DKIM signing active (test email passes DMARC)	Notifications	
2	Twilio SMS delivers to Indian numbers with DLT-approved template	Notifications	
3	FCM push notifications deliver to Android + iOS test devices	Notifications	
4	Google Calendar events created on appointment booking	Notifications	

STEP 16

Monitoring + Analytics

Deploy Prometheus, Grafana, and Sentry; create the four golden dashboards.

Observability is non-negotiable for production. Deploy Prometheus + Grafana via the Helm charts in monitoring/; configure the backend to expose a /metrics endpoint with the prom-client library (request rate, error rate, latency histogram, active connections). Build four dashboards in Grafana: **API Health** (RED

metrics — Rate, Errors, Duration), **Database** (connection pool, slow queries, replication lag), **AI Quality** (token usage, tool-call accuracy, hallucination rate, latency), and **Business** (revenue, orders, calls, conversion funnel). Add Sentry for error tracking — the SDK is already a dependency, just set `SENTRY_DSN`.

Acceptance checkpoints

#	Checkpoint	Owner	Done
1	Prometheus scrapes backend /metrics every 15s without errors	DevOps	
2	Four Grafana dashboards created and shared with the team	DevOps	
3	Sentry receives errors from backend + all four frontend apps	DevOps	
4	Alert rules: 5xx rate > 1%, p95 latency > 500ms, DB connections > 80%	DevOps	

STEP 17

Security

Fix the three critical vulnerabilities and pass an external security scan.

The `PRODUCTION_READINESS_ROADMAP.md` identifies three critical vulnerabilities that must be fixed before any production traffic: (1) the RDS security group is open to `0.0.0.0/0` on port 5432 — restrict ingress to the EKS node group security group only; (2) the Kubernetes Secret contains a plaintext `SECRET_KEY: CHANGE_ME_IN-production` — adopt the External Secrets Operator + AWS Secrets Manager pattern; (3) OAuth2 codes and refresh tokens are stored in process memory, which breaks under multi-replica load balancing — move to Redis with TTL and implement a JTI blocklist for JWT revocation.

Beyond the three critical fixes, complete the medium-severity items: add per-user rate limiting (Redis sliding window), enforce webhook signature verification unconditionally (remove the dev bypass), add CSRF protection for cookie-based auth flows, integrate Microsoft Presidio for PII redaction in logs, add Pod SecurityContext (runAsNonRoot, readOnlyRootFilesystem, drop ALL capabilities), add PodDisruptionBudgets (minAvailable: 2), and create dedicated ServiceAccounts with IRSA per service. Run an external security scan (AWS Inspector, Snyk, OWASP ZAP) and confirm no Critical or High findings.

Compliance note: Dayjoy processes customer PII (name, phone, address, health queries). Under India's DPDP Act 2023, you must publish a privacy policy, obtain explicit consent, allow data export and deletion, and maintain an audit log with hash chaining. The audit log table is specified in the schema but the chaining logic is a stub — implement it before launch.

Acceptance checkpoints

#	Checkpoint	Owner	Done
1	RDS security group ingress restricted to EKS node SG only	DevOps	
2	All K8s secrets sourced from AWS Secrets Manager via ExternalSecrets	DevOps	
3	OAuth codes + refresh tokens in Redis; JWT JTI blocklist active	Backend	
4	External security scan (Snyk + Inspector) passes with no Critical/High	Security	

STEP 18

Testing

Run the full test suite (unit + integration + e2e + load) and achieve ≥ 80% coverage.

The repository ships with a comprehensive test suite under `testing/`: unit tests (Vitest), integration tests (per module), API tests (Supertest), e2e tests (Playwright), RAG evaluation tests, AI evaluation tests, and a Locust load test. Run them in this order, fixing failures as you go. The minimum acceptable coverage is 80% for the backend and 70% for the frontend. The RAG precision@5 must be ≥ 0.85 and the AI tool-selection accuracy must be ≥ 88%.

```
# Unit + integration
pnpm test

# End-to-end
pnpm test:e2e

# Load test (requires backend running on :3000)
python testing/load/load-test.py --base-url http://localhost:3000 --users 50 --duration 5m

# Coverage report
pnpm --filter backend test:coverage
```

Acceptance checkpoints

#	Checkpoint	Owner	Done
1	pnpm test passes; backend coverage ≥ 80%, frontend ≥ 70%	QA	
2	pnpm test:e2e passes; all critical user journeys green	QA	
3	Load test: p95 latency < 500ms at 50 concurrent users, 0 errors	QA	
4	RAG precision@5 ≥ 0.85; AI tool-selection accuracy ≥ 88%	AI	

STEP 19

Staging Deployment

Deploy to the staging Kubernetes cluster via Helm and verify all services are healthy.

With all tests passing, deploy to the staging cluster. The deployment manifests live in `deployment/kubernetes/` and use Helm charts for parameterized rollout. Apply in this order: namespace, external-secrets operator, postgres-operator, redis-operator, the backend deployment, the four frontend deployments, n8n, monitoring stack, and ingress. Verify each deployment reaches a Ready state before moving to the next. The ingress should terminate TLS at the load balancer (AWS ALB) and route based on hostname: `api.dayjoy.ai` → backend, `admin.dayjoy.ai` → admin-dashboard, etc.

```
# Apply Helm charts to staging
helm upgrade --install dayjoy-backend ./deployment/helm/backend \
  -f ./deployment/helm/backend/values.staging.yaml \
  --namespace dayjoy-staging --create-namespace

# Verify all pods are Ready
kubectl get pods -n dayjoy-staging

# Verify ingress
kubectl get ingress -n dayjoy-staging
curl https://api.staging.dayjoy.ai/health | jq
```

Acceptance checkpoints

#	Checkpoint	Owner	Done
1	All pods in dayjoy-staging namespace are Ready (1/1) and not restarting	DevOps	
2	Ingress routes correctly; TLS certs valid (Let's Encrypt or ACM)	DevOps	
3	ExternalSecrets synced all secrets from AWS Secrets Manager	DevOps	
4	Staging URL accessible from public internet; /health returns 200	DevOps	

STEP 20

Real User Pilot

Onboard 5-10 friendly users, run for 7 days, and collect structured feedback.

Before opening the floodgates, run a closed pilot with 5-10 friendly users spanning all three personas (1-2 customers, 2-3 distributors, 2-3 employees, 1 admin). Give them staging credentials and a structured task list covering the critical journeys: place an order, raise a support ticket, book an appointment, call Sarah, chat on WhatsApp, view analytics. Collect feedback via a Typeform survey after 7 days, focusing on: task completion rate, time-on-task, CSAT (1-5), and qualitative friction points. Triage every P0/P1 bug before proceeding to production.

Acceptance checkpoints

#	Checkpoint	Owner	Done
1	5-10 pilot users onboarded across all three personas	Product	
2	7-day pilot completed; CSAT ≥ 4.0/5	Product	
3	All P0 bugs fixed; P1 bugs triaged with owners	Engineering	
4	Pilot retrospective doc published; go/no-go decision recorded	Product	

STEP 21

Production Deployment

Cut over to the production cluster with blue-green rollout and DNS cutover.

Production deployment uses a blue-green strategy: deploy the new version to the green cluster, run smoke tests against it, switch the ALB target group from blue to green, and keep the blue cluster warm for 24 hours as a rollback target. Apply the production Helm values (values.prod.yaml) which set higher replica counts (backend: 4, frontend: 2 per app), larger resource limits, and production-grade secrets from AWS

Secrets Manager. Cut DNS from staging.dayjoy.ai to dayjoy.ai at a low-traffic time (recommended: 02:00 IST Sunday).

```
# Deploy to production green cluster
helm upgrade --install dayjoy-backend ./deployment/helm/backend \
  -f ./deployment/helm/backend/values.prod.yaml \
  --namespace dayjoy-prod --create-namespace

# Run smoke tests against green
bash deployment/scripts/verify.sh --target green

# Switch ALB target group (blue → green)
aws elbv2 modify-target-group-attributes ...

# DNS cutover (Route 53 weighted routing: 0% blue, 100% green)
aws route53 change-resource-record-sets ...
```

Acceptance checkpoints

#	Checkpoint	Owner	Done
1	Green cluster deployed; all pods Ready; smoke tests pass	DevOps	
2	ALB target group switched blue → green; traffic now flows to green	DevOps	
3	DNS cutover complete; https://api.dayjoy.ai/health returns 200	DevOps	
4	Blue cluster kept warm for 24h; then scaled to 0 (rollback target)	DevOps	

STEP 22

Production Verification

Run end-to-end production verification across all channels and confirm SLOs are met.

Within 24 hours of the production cutover, run a structured verification pass covering every channel and every persona. Place a real order via the customer portal, call Sarah on the production Vapi number, send a WhatsApp message to the business number, chat on the live website, and have an employee log in and assign a lead. For each channel, verify: (a) the request reaches the backend (visible in CloudWatch logs and Grafana API Health dashboard), (b) the AI reply is correct and grounded (visible in the AI Quality dashboard), (c) the conversation appears in the admin dashboard, and (d) no error is logged in Sentry. Compare observed SLOs against targets: p95 latency < 500ms, 99.9% uptime, AI accuracy ≥ 92%, CSAT ≥ 4.5.

If any SLO is missed, do not roll back immediately — first check whether the issue is configuration (fixable in-place) or architectural (requires rollback). Common post-launch issues: CDN cache misses on static assets (purge and re-warm), database connection pool exhaustion (increase pool size or add PgBouncer), OpenAI rate limits (request a quota increase or add a fallback model), and WhatsApp template rejections (resubmit with corrected content). Document every issue and its resolution in the post-launch retrospective.

Sign-off: Once all four SLOs are met for 7 consecutive days, the platform is declared production-stable. Schedule a weekly ops review (Mondays 10:00 IST) to monitor SLOs, review Sentry trends, and triage incoming bugs. Set a quarterly security review and an annual disaster-recovery drill.

Acceptance checkpoints

#	Checkpoint	Owner	Done
1	End-to-end verification across all 4 channels (voice, WhatsApp, web, portal) passes	QA	
2	SLOs met for 7 consecutive days: p95 < 500ms, 99.9% uptime, AI ≥ 92%, CSAT ≥ 4.5	DevOps	
3	Sentry shows no new errors in 24h; all critical alerts resolved	DevOps	
4	Post-launch retrospective doc published; weekly ops review scheduled	Product	

Appendix — Effort & Timeline Summary

The table below summarizes the expected effort and ownership for each step, assuming a team of 3-4 engineers (1 backend, 1 frontend, 1 DevOps, 1 part-time AI/data). Total elapsed time is approximately 10-12 weeks for a focused team; double that for a part-time team. Steps 5, 6, and 17 are the longest poles and should be started as early as possible.

Step	Name	Effort	Owner
1	Environment & Secrets	0.5 day	DevOps
2	Local Infrastructure	0.5 day	DevOps
3	Database	1 day	Backend
4	Backend	1 day	Backend
5	RAG + Knowledge	5-7 days	AI
6	AI / LLM	5-7 days	AI
7	Vapi Voice AI	3-5 days	Voice
8	WhatsApp AI	2-3 days	WhatsApp
9	Website AI Chat	1-2 days	Frontend
10	Customer Portal	5-7 days	Frontend
11	Distributor Portal	5-7 days	Frontend
12	Employee Portal	5-7 days	Frontend
13	Admin Dashboard	2-3 days	Frontend
14	n8n Automation	3-5 days	Automation
15	Notifications + Email + Calendar	3-5 days	Backend
16	Monitoring + Analytics	2-3 days	DevOps
17	Security	5-7 days	Security
18	Testing	3-5 days	QA

19	Staging Deployment	1-2 days	DevOps
20	Real User Pilot	7 days (elapsed)	Product
21	Production Deployment	1 day	DevOps
22	Production Verification	7 days (elapsed)	QA

Mentor's closing note: The Dayjoy AI Enterprise codebase is approximately 70% production-ready. The 22 steps in this guide close the remaining 30%. Resist the urge to skip ahead — every step depends on the artifacts produced by the previous one, and every shortcut will surface as a production incident within the first week of pilot. Move deliberately, document every decision in your worklog, and run the acceptance checklist religiously. You will reach production-stable in 10-12 weeks.